

FLAWFIX SECUREASSIST

Automated Vulnerability Detection, Semantic LLVM IR Analysis & Patch Verification Audit Report

Target File: pytest.py_patched	Language: PYTHON
Project: FlawFix Workspace	Audit Date: 2026-10-06 22:10:45
Report ID: RPT-90F6C7B7	Engine: FlawFix LLVM+XAI v1.0

1. Executive Security Assessment

The provided Python code correctly utilizes parameterized queries for SQLite interaction, which is the industry-standard defense against SQL Injection (CWE-89). The input 'username' is passed as a separate tuple argument to the 'execute' method, ensuring the database driver treats the input as data rather than executable SQL code.

TOTAL FLAWS	CRITICAL	HIGH	MEDIUM	LOW
0	0	0	0	0

2. Detected Vulnerability Inventory

✓ No security vulnerabilities detected in this source code.

4. Automated Patch Verification Audit

Patch for VULN-001: **VERIFIED & COMPILER-VALIDATED**

The original code was vulnerable to SQL injection by directly concatenating user input into the query string. The patch correctly implements parameterized queries using the '?' placeholder, which ensures that the database driver treats the input as data rather than executable SQL code, effectively neutralizing the injection vector.

5. AI Code Optimization Recommendations

- Use Context Managers for Database Connections:** Utilize the 'with' statement for sqlite3 connections to ensure that the connection is automatically closed even if an exception occurs during execution. *(Impact: Improves resource management and prevents potential connection leaks.)*
- Input Sanitization:** While parameterized queries prevent SQL injection, adding basic length validation on the 'username' input can prevent potential Denial of Service (DoS) attacks involving excessively large strings. *(Impact: Enhances application robustness against malformed or malicious input sizes.)*